

HOW THE ROLLING-ORIGIN PARTITIONS OF THE bCCNN ARE USED

The three time-aware partitions compared with K-fold cross-validation

Study notes on the bCCNN strand of the research assignment

Prepared with the generative artificial intelligence tool Claude (Anthropic)
as study material for the BComHons research assignment
Department of Statistics and Actuarial Science
Stellenbosch University

Not for submission for assessment

October 2026

Note on the preparation of these notes

These notes were prepared with the generative artificial intelligence (AI) tool Claude (Anthropic) at the request of the student, as study material on the blended cross-classified neural network (bCCNN) strand of the research assignment. They explain the mathematics of the model, the code in the student's repository (Masters-Research-R) and the literature that the code follows.

The notes are not the student's original work and are not intended for submission for assessment. Section 2.6.3 of the Research Assignment Guide (Department of Statistics and Actuarial Science, 2026: 7-8) prohibits the use of AI tools to generate content that is submitted for assessment and requires a declaration of any use of such tools. For this reason the plagiarism declaration and the declaration on the use of artificial intelligence tools of the Department template are not included. The layout follows the technical requirements in Chapter 5 of the Guide and the Department template.

Abstract

The blended cross-classified neural network is, by default, early-stopped on a time-aware split of the observed claims triangle into three rolling-origin partitions, with valuation periods 15, 18 and 20. This report sets out exactly what the code computes on each partition, which of the results reach the reported reserve and which are only reported, and whether the procedure is a form of K-fold cross-validation.

The answer is that it is not. Only the final partition chooses the number of gradient-descent steps of the reported model, from its own validation curve and without any averaging over partitions. The two earlier partitions each choose their own number of steps, replay the whole procedure at their earlier valuation date and score it on the calendar periods that followed; their deviances are pooled into a test error that the code reports but does not optimise. The held-out sets of the partitions overlap, they cover fewer than half of the observed cells, and they always lie later in calendar time than the cells that are fitted. Each of these properties contradicts one defining property of K-fold cross-validation, which uses disjoint folds that cover every observation once and averages the K scores to make a choice.

The report proves that a K-fold split of the cells of the triangle cannot be combined with the chain-ladder start, because the latest accident period and the latest development period each consist of a single cell. It relates the test partitions to time-series cross-validation, the claims split of Paper C to a single hold-out, and the reversed claims split of Härkönen (2021) to a two-fold arrangement that averages predictions. A replication in Python confirms the overlaps and the coverage counts.

Key words: Claims reserving; Neural networks; Early stopping; Rolling-origin validation; Time-series cross-validation; K-fold cross-validation; Out-of-time testing; Chain ladder

Table of contents

Note on the preparation of these notes.....	ii
Abstract.....	iii
List of tables.....	vi
List of figures.....	vii
List of abbreviations and/or acronyms.....	viii
CHAPTER 1 INTRODUCTION.....	1
1.1 INTRODUCTION.....	1
1.2 THE QUESTION AND THE SHORT ANSWER.....	1
1.3 SCOPE AND LIMITATIONS.....	2
1.4 HOW IT WAS CHECKED.....	2
1.5 CHAPTER OUTLINE.....	3
CHAPTER 2 THE THREE PARTITIONS AND HOW EACH IS USED.....	4
2.1 INTRODUCTION.....	4
2.2 THE SETS OF A PARTITION.....	4
2.3 THE COMPUTATION ON EVERY PARTITION.....	4
2.4 THE TWO TEST PARTITIONS: REPLAY AND SCORE.....	5
2.5 THE POOLED TEST ERROR.....	6
2.6 THE FINAL PARTITION: THE NUMBER OF STEPS OF THE REPORTED MODEL.....	6
2.7 THE PROCEDURE AS ONE MAP.....	7
2.8 SUMMARY.....	7
CHAPTER 3 K-FOLD AND TIME-SERIES CROSS-VALIDATION.....	8
3.1 INTRODUCTION.....	8
3.2 K-FOLD CROSS-VALIDATION.....	8
3.3 K-FOLD CROSS-VALIDATION OF THE NUMBER OF STEPS.....	8
3.4 TIME-SERIES CROSS-VALIDATION.....	9
3.5 SUMMARY.....	10
CHAPTER 4 THE ROLLING ORIGIN COMPARED WITH K-FOLD CROSS-VALIDATION.....	11

4.1	INTRODUCTION.....	11
4.2	PROPERTY BY PROPERTY.....	11
4.2.1	The held-out sets overlap.....	11
4.2.2	The held-out sets do not cover the triangle.....	11
4.2.3	The roles are ordered in time.....	12
4.2.4	The choice is not averaged.....	12
4.2.5	The held-out cells are not like the rest.....	12
4.2.6	Summary of the comparison.....	12
4.3	WHY K-FOLD CROSS-VALIDATION OVER THE CELLS IS INFEASIBLE.....	13
4.4	THE CLAIMS SPLIT AND TWO-FOLD CROSS-VALIDATION.....	14
4.5	WHAT THE PROCEDURE SHOULD BE CALLED.....	14
4.6	SUMMARY.....	15
	CHAPTER 5 NUMERICAL ILLUSTRATION AND CONCLUSIONS.....	16
5.1	INTRODUCTION.....	16
5.2	WHERE THE HELD-OUT CELLS GO.....	16
5.3	SUMMARY AND CONCLUSIONS.....	18
	References.....	20
	APPENDIX A LIST OF SYMBOLS.....	21

List of tables

Table 2.1 Sizes of the sets of the three partitions.....	4
Table 2.2 What each partition chooses and where the result goes.....	7
Table 4.1 The procedure of the code against K-fold and time-series cross-validation.....	13
Table 5.1 Roles in later partitions of the cells held out by each partition.....	16
Table A.1 Symbols used in this report.....	21

List of figures

Figure 5.1 Roles of the cells of each calendar period: rolling origin against five-fold cross-validation.....	17
Figure 5.2 Held-out cells over the triangle: five-fold cross-validation against the rolling origin.....	18

List of abbreviations and/or acronyms

bCCNN	Blended cross-classified neural network
ccODP	Cross-classified over-dispersed Poisson (model)
CV	Cross-validation
MDN	Mixture density network
RMSprop	Root mean square propagation (optimiser)

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

This report is the fifth of a set of explanatory texts on the blended cross-classified neural network (bCCNN), a network that starts from the cross-classified over-dispersed Poisson (ccODP) model, as implemented in the code of the repository. The texts are referred to as companions. Companion I is *The mathematics behind the bCCNN: the ccODP model, the network and its chain-ladder start*; companion II is *The fitting procedure and training of the bCCNN*; companion III is *The bCCNN as a whole: interpretation, evaluation and uncertainty*; companion IV is *The time-aware split for the bCCNN: rolling-origin training, validation and test partitions*; and companion V is the present report.

Companion IV constructs the time-aware split cell by cell. The configuration file `config.yml` sets `validation: rolling_origin`, so `rolling_origin_sets()` in `R/triangles.R` cuts the observed 20×20 triangle into three partitions, with valuation periods $\tau = 15, 18$ and 20 , after Al-Mudafer, Avanzi, Taylor and Wong (2021). Each partition has training cells, validation cells and, for the two earlier partitions, test cells. The present report takes that construction as given and answers a narrower question: how exactly are the three partitions *used*, and is the procedure a form of K-fold cross-validation (CV)?

The question arises naturally. The procedure fits the same model several times on different subsets of the same triangle and scores each fit on cells it did not train on, which is how K-fold CV is usually described. The configuration file also contains an entry `cv: folds: 5`. The report shows that the resemblance is superficial: the procedure is a rolling-origin, or forward-chaining, design, and the entry `cv: folds: 5` is read by no function of the code.

1.2 THE QUESTION AND THE SHORT ANSWER

The question addressed is whether the three rolling-origin partitions are used as the folds of a K-fold CV, and, if not, what each partition contributes to the reported model. The short answer, which Chapters 2 to 5 establish, has five parts.

- i) The number of gradient-descent steps of the reported model is chosen on the final partition alone ($\tau = 20$): it is the step with the lowest deviance on the validation cells of that partition. Nothing is averaged over partitions.
- ii) Each of the two test partitions ($\tau = 15$ and $\tau = 18$) chooses its own number of steps on its own validation cells, but that number is used only to replay the procedure at the earlier valuation date. The replayed fit is scored on the calendar periods after τ , and the two scores are pooled into a test error that the code reports and never minimises.

- iii) The held-out sets of the partitions are not disjoint, they do not cover the triangle, and they always lie later in calendar time than the cells that are fitted. K-fold CV requires disjoint folds that cover every observation exactly once, and treats the folds symmetrically.
- iv) A K-fold split of the cells of the triangle is not merely different but infeasible with the chain-ladder start, because one fold must hold out the single cell of the latest accident period and another the single cell of the latest development period.
- v) The design is a rolling-origin evaluation, which Hyndman and Athanasopoulos (2021) call time-series cross-validation, combined with a single out-of-time hold-out that chooses the number of steps. The claims split of Paper C is a single hold-out as well; only the reversed claims split of Härkönen (2021) resembles a two-fold design.

1.3 SCOPE AND LIMITATIONS

The report explains the use of the partitions in `bccnn_ro_partition()`, `bccnn_rolling_origin()` and `bccnn_calibrate()` in `R/nn_models.R`, with the configuration of the code: $n=20$, `test_periods = c(5, 2)`, `vali_periods = 2`, `exclude = 2`, `final_fit = "refit"` and $T_{max}=1\,000$ steps. The alternative `final_fit = "partition"` is mentioned where it changes the answer. The statements about Al-Mudafer *et al.* (2021) and about Gabrielli, Richman and Wüthrich (2020), referred to as Paper C as in the code, rest on the quotations in the literature notes in the repository (thesis/Hyperparameter search), which cite the papers by PDF page; Paper C is quoted from its SSRN version (Gabrielli, Richman and Wüthrich, 2018). These notes have not rechecked the quotations against the papers themselves. The quotations of Härkönen (2021) were checked against the text of the thesis.

No network was trained for this report. The numbers of steps, deviances and test errors depend on the data and the seed, and none are reported. All counts in the report are properties of the split and hold for any data.

1.4 HOW IT WAS CHECKED

The partitions of `rolling_origin_sets()` were rebuilt in Python from their definition (companion IV, Section 2.3), for the configuration of the code, in the script `kfold_check_guide.py` kept with these explainers. The script counts how the cells of each partition move between roles in the later partitions, how often each cell is held out, and, for a random five-fold split of the cells, which accident and development periods lose all their training cells. The overlap and coverage counts of Chapter 4 were derived by hand and agree with the script; the figures of Chapter 5 are drawn from its output. The use of each partition was read from the code line by line.

1.5 CHAPTER OUTLINE

Chapter 2 states what the code computes on each partition and where each result goes, ending with the procedure as one map. Chapter 3 defines K-fold CV, its early-stopping version and time-series CV. Chapter 4 compares the rolling origin with K-fold CV property by property, proves that K-fold CV over the cells is infeasible with the chain-ladder start, places the claims split in the same frame, and names the design. Chapter 5 gives the numerical illustration and the conclusions. The symbols are listed in Appendix A.

CHAPTER 2

THE THREE PARTITIONS AND HOW EACH IS USED

2.1 INTRODUCTION

Chapter 1 posed the question. This chapter answers its first half from the code: it recalls the sets that make up a partition, states the computation run on every partition, follows the two different uses of that computation (the test partitions and the final partition), and summarises the flow of information from the partitions to the reported reserve.

2.2 THE SETS OF A PARTITION

Cell (i, j) of the triangle lies in accident period i , development period j and calendar period $k = i + j - 1$. The observed triangle at the valuation date $n = 20$ is $D = \{(i, j) : k \leq n\}$, with 210 cells; the cells with $k > n$ form the lower triangle F , whose sum is the reserve. For each valuation period $\tau \in \{15, 18, 20\}$, `rolling_origin_sets()` returns the triangle that would have been observed at τ , split into training and validation cells, and the test cells that follow it (companion IV, (2.1) to (2.5)):

$$\begin{aligned} D_\tau &= \{(i, j) : 1 \leq i, j \leq \tau, k \leq \tau\} = U_\tau \cup V_\tau, U_\tau \cap V_\tau = \emptyset, \\ T_\tau &= \{(i, j) : 1 \leq i, j \leq \tau, \tau < k \leq n\}, T_{20} = \emptyset. \end{aligned} \quad (2.1)$$

The validation cells V_τ are the two latest diagonals of D_τ without the first two accident and development periods, plus two replacement cells at development period 2; the training cells are $U_\tau = D_\tau \setminus V_\tau$. Table 2.1 repeats the sizes from companion IV, Table 2.1. Within one partition the three sets are disjoint and play fixed roles; the question of this report concerns how the sets of *different* partitions relate and how their results are combined.

Table 2.1 **Sizes of the sets of the three partitions**

Partition	τ	Training $ U_\tau $	Validation $ V_\tau $	Test $ T_\tau $	Calendar periods tested
Test partition 1	15	97	23	60	16 to 20
Test partition 2	18	142	29	33	19, 20
Final partition	20	177	33	0	None

2.3 THE COMPUTATION ON EVERY PARTITION

`bccnn_rolling_origin()` calls `bccnn_ro_partition()` once for each partition, in the order $\tau = 15, 18, 20$, and the calls do not depend on one another: no result of one partition is passed to the next. Each call runs the same three steps.

The start. The ccODP is fitted on the training cells only, $\hat{\vartheta}(U_\tau)$, by `fit_odp_glm(part$y, cells = part$train)`, and the network is initialised at the ccODP start $\theta_0(\hat{\vartheta}(U_\tau))$ of companion I.

The run. From that start, T_{max} full-batch RMSprop steps with dropout are taken on the training cells,

$$\theta_t^{(\tau)} = T_t^s(\theta_0(\hat{\vartheta}(U_\tau)), U_\tau), t = 0, 1, \dots, T_{max}, \quad (2.2)$$

where $T_t^s(\theta, S)$ denotes t steps with seed s on the cells S (companion II, Section 4.4). After every step the deviance without dropout is recorded on the validation cells and, for a test partition, on the test cells.

The choice of the step. The step with the lowest validation deviance is kept,

$$\begin{aligned} D_{V,\tau}(t) &= D(y, \mu(\theta_t^{(\tau)}); V_\tau), \\ t_\tau^* &= \arg \min_{0 \leq t \leq T_{max}} D_{V,\tau}(t), \end{aligned} \quad (2.3)$$

in the code `best <- h$epoch[which.min(h$vali)]`. Validation cells in a period without payments in the training cells are left out of $D_{V,\tau}$ (companion IV, Section 3.3).

Each partition therefore produces its own number of steps t_τ^* , chosen on its own validation cells. What happens to t_τ^* depends on whether the partition is a test partition or the final one.

2.4 THE TWO TEST PARTITIONS: REPLAY AND SCORE

A test partition imitates the reserving problem at the earlier valuation date τ and asks how well the whole procedure would have forecast the $n - \tau$ calendar periods that followed. With `final_fit = "refit"` the procedure is replayed exactly as it is applied at n : the chain ladder of all cells observed at τ gives the start, the network is trained on all of D_τ for the t_τ^* steps chosen in (2.3), and the result is scored on the test cells,

$$\begin{aligned} \theta_\tau^{refit} &= T_{t_\tau^*}^s(\theta_0(\hat{\vartheta}(D_\tau)), D_\tau), \\ S_\tau &= D(y, \mu(\theta_\tau^{refit}); T_\tau), \tau \in \{15, 18\}. \end{aligned} \quad (2.4)$$

The same test cells are scored for the chain ladder at τ , $\hat{\vartheta}(D_\tau)$, and for the ccODP on the training cells, $\hat{\vartheta}(U_\tau)$, so that each test partition gives three test deviances (companion IV, Section 3.4). With `final_fit = "partition"` the network $\theta_{t_\tau^*}^{(\tau)}$ of (2.2) is scored instead, and no refit is run.

Two features of (2.4) matter for the comparison with K-fold CV. First, t_τ^* is used inside the partition only: it fixes the length of the replayed fit at τ and is then discarded. Second, the test cells are scored after the choice: the test deviance on T_τ is recorded along the run of (2.2), but (2.3) never reads it.

2.5 THE POOLED TEST ERROR

`bccnn_rolling_origin()` pools the scores of the two test partitions into one number per model, the test deviance per test cell of equation (4.4) of Al-Mudafer *et al.* (companion IV, (3.4)):

$$\dot{D} = \frac{S_{15} + S_{18}}{|T_{15}| + |T_{18}|} = \frac{S_{15} + S_{18}}{60 + 33}. \quad (2.5)$$

This is an average over the test partitions, and in that one respect it resembles a CV score. It differs from one in use: the code computes $\dot{D}$ for the bCCNN, for the chain ladder at τ and for the ccODP on the training cells, prints them in `bccnn_tables()` as the row "test error (4.4)", and stops there. No quantity of the code is chosen to minimise $\dot{D}$; the architecture, the dropout rate and the learning rate are fixed in `config.yml`.

In Al-Mudafer *et al.* (2021) the same average is the selection criterion. Their hyperparameters are chosen one at a time on it: "Select the coefficient with the lowest test error" (Al-Mudafer *et al.*, 2021: 12), and the number of mixture components "is increased so long as the test error decreases" (Al-Mudafer *et al.*, 2021: 11). Each hyperparameter set is scored as an average over several runs: "the MDN is trained 2T times for each set of hyper-parameters θ , as each run has a different weight initialisation and hence a different fit. Averaging the error of these runs reduces the impact of random weight initialisations on the performance of the hyper-parameter set θ " (Al-Mudafer *et al.*, 2021: 16). Their test partitions therefore act as the validation folds of a time-series CV for the hyperparameters, and their final model is trained on the third partition: "MDN with hyper-parameters $\theta_{\min}$ is fit 5 times on the training data of Partition 3, under different weight initialisations. The 5 fitted distributions are ensembled to produce the final forecast" (Al-Mudafer *et al.*, 2021: 17). The code keeps the partitions and the pooled error of Al-Mudafer *et al.*, but not the search that minimises it.

2.6 THE FINAL PARTITION: THE NUMBER OF STEPS OF THE REPORTED MODEL

`bccnn_calibrate()` takes the final partition, `val <- ro$final`, and uses its number of steps, `steps <- val$best_epoch`, unless `epochs` overrides it. With `final_fit = "refit"` the reported model is the network trained on the whole observed triangle for that number of steps from the chain ladder of D :

$$\begin{aligned} t^* &= t_{20}^* = \arg \min_{0 \leq t \leq T_{\max}} D_{V,20}(t), \\ \hat{\theta} &= T_{t^*}^s(\theta_0(\hat{\vartheta}(D)), D), \hat{R} = \sum_{(i,j) \in F} \mu_{i,j}(\hat{\theta}). \end{aligned} \quad (2.6)$$

With `final_fit = "partition"` the network of the final partition at step t_{20}^* is reported instead, without a refit. In both cases the choice rests on the 33 validation cells of the final partition: the latest two diagonals, $k = 19$ and 20 , without the first two accident and development periods, plus the cells $(6, 2)$ and $(14, 2)$. The numbers t_{15}^* and t_{18}^* do not enter t^* , and neither do the test scores S_{15} , S_{18} or $\dot{D}$.

2.7 THE PROCEDURE AS ONE MAP

The flow of information can be written as two separate chains that share only the data:

$$\begin{aligned} (U_{20}, V_{20}) &\rightarrow t_{20}^* \rightarrow \hat{\theta} \rightarrow \hat{R}, \\ (U_{\tau}, V_{\tau}) &\rightarrow t_{\tau}^* \rightarrow \theta_{\tau}^{refit} \rightarrow S_{\tau} \rightarrow \dot{D}, \tau = 15, 18. \end{aligned} \quad (2.7)$$

The first chain produces the reserve; the second produces a diagnostic. In steps, the default procedure is as follows.

- i) Build the three partitions with `rolling_origin_sets()`.
- ii) For $\tau = 15$: fit the ccODP on U_{15} , run (2.2), choose t_{15}^* by (2.3), refit on D_{15} for t_{15}^* steps and score on the 60 test cells, (2.4).
- iii) For $\tau = 18$: the same, with t_{18}^* and the 33 test cells.
- iv) For $\tau = 20$: fit the ccODP on U_{20} , run (2.2) and choose t_{20}^* by (2.3); there is nothing to score.
- v) Pool the test scores of steps ii) and iii) into $\dot{D}$, (2.5), and report it.
- vi) Fit the chain ladder on D and train the reported network for t_{20}^* steps, (2.6).

Table 2.2 summarises the role of each partition.

Table 2.2 What each partition chooses and where the result goes

Partition	Chooses	On which cells	Result used for	Enters the reported reserve
$\tau = 15$	t_{15}^*	V_{15} : 23 cells, mostly $k = 14, 15$	Length of the refit at 15, scored on 60 test cells; part of $\dot{D}$	No
$\tau = 18$	t_{18}^*	V_{18} : 29 cells, mostly $k = 17, 18$	Length of the refit at 18, scored on 33 test cells; part of $\dot{D}$	No
$\tau = 20$	t_{20}^*	V_{20} : 33 cells, mostly $k = 19, 20$	Number of steps t^* of the reported model	Yes
All test partitions	Nothing	$T_{15} \cup T_{18}$	The reported test error $\dot{D}$	No

2.8 SUMMARY

Every partition runs the same computation: a ccODP start on its training cells, an early-stopping run and a choice of the number of steps on its own validation cells. The final partition's number of steps becomes the number of steps of the reported model. The test partitions use their numbers of steps only to replay the procedure at their earlier valuation dates and score it on the calendar periods that followed; their scores are pooled into a test error that is reported and not minimised. Nothing is averaged to make a choice. Chapter 3 defines K-fold CV so that the two designs can be compared precisely.

CHAPTER 3

K-FOLD AND TIME-SERIES CROSS-VALIDATION

3.1 INTRODUCTION

Chapter 2 described what the code does. This chapter defines the two procedures it might be mistaken for: K-fold CV, in its general form and in the form it would take for the number of steps of the bCCNN, and time-series CV, the rolling-origin evaluation of the forecasting literature.

3.2 K-FOLD CROSS-VALIDATION

Let I be the index set of the observations. K-fold CV splits I into K folds of roughly equal size, usually at random (Hastie, Tibshirani and Friedman, 2009: 241-242):

$$I = F_1 \cup F_2 \cup \dots \cup F_K, F_k \cap F_l = \emptyset (k \neq l), |F_k| \approx |I|/K. \quad (3.1)$$

For a tuning parameter λ , the model is fitted K times, the k -th time on all folds except F_k , giving $\hat{f}_\lambda^{(-k)}$, and each fit is scored on the fold it did not see. The CV estimate of the prediction error is the average loss over all observations, each scored by the fit that left it out:

$$\begin{aligned} CV(\lambda) &= \frac{1}{|I|} \sum_{k=1}^K \sum_{i \in F_k} L(y_i, \hat{f}_\lambda^{(-k)}(x_i)), \\ \hat{\lambda} &= \arg \min_{\lambda} CV(\lambda). \end{aligned} \quad (3.2)$$

The model is then refitted on all of I with $\hat{\lambda}$. Five properties define the procedure, and they are used in Chapter 4.

- (K1) The folds are disjoint.
- (K2) The folds cover I : every observation is held out exactly once and used for fitting in the other $K - 1$ fits.
- (K3) The folds play symmetric roles: no fold is special, and any fold could be listed first.
- (K4) The choice is made on the average (3.2) over all K fits, not on one of them.
- (K5) The held-out fold is assumed to be like the data to be predicted: with exchangeable observations, $CV(\lambda)$ estimates the error at a new draw from the same distribution.

3.3 K-FOLD CROSS-VALIDATION OF THE NUMBER OF STEPS

For the bCCNN the tuning parameter would be the number of steps t . A K-fold CV over the cells of the observed triangle would fit the ccODP and run the network K times, each time without one fold, and average the validation curves before choosing:

$$\begin{aligned}
\theta_t^{(-k)} &= T_t^s \left(\theta_0 \left(\hat{\vartheta} \left(D \setminus F_k \right) \right), D \setminus F_k \right), \\
CV(t) &= \frac{1}{|D|} \sum_{k=1}^K D \left(y, \mu \left(\theta_t^{(-k)} \right); F_k \right), \\
\hat{t} &= \arg \min_{0 \leq t \leq T_{\max}} CV(t),
\end{aligned} \tag{3.3}$$

followed by the refit (2.6) with $\hat{t}$ in place of t_{20}^* . An alternative averages the K minimisers instead of the curves. The entry `cv: folds: 5` of `config.yml` would suggest this design with $K=5$, but no function of the code reads `cfg$cv`; the procedure of Chapter 2 is the only one implemented. Section 4.3 shows that (3.3) cannot be computed for any $K \geq 2$.

3.4 TIME-SERIES CROSS-VALIDATION

When the observations are ordered in time and the aim is to forecast, the forecasting literature replaces the random folds by a sequence of forecast origins. At each origin the model is fitted on the observations up to the origin and scored on the observations after it, and the scores are averaged over the origins. This is the 'rolling forecasting origin' reviewed by Tashman (2000), which Hyndman and Athanasopoulos (2021) call time-series cross-validation. With origins $\tau_1 < \dots < \tau_m$, training sets D_{τ_r} and test sets T_{τ_r} after each origin, the score and the choice are

$$\begin{aligned}
TSCV(\lambda) &= \frac{\sum_{r=1}^m D \left(y, \mu_{\lambda}^{(\tau_r)}; T_{\tau_r} \right)}{\sum_{r=1}^m |T_{\tau_r}|}, \\
\hat{\lambda} &= \arg \min_{\lambda} TSCV(\lambda).
\end{aligned} \tag{3.4}$$

Time-series CV keeps (K4), the average over fits, but gives up (K1) to (K3): the training sets are nested and expanding, $D_{\tau_1} \subset \dots \subset D_{\tau_m}$, the test sets of different origins overlap, observations before the first origin are never tested, and the origins are ordered in time. It replaces (K5) by the assumption that what is learned before an origin carries over to the periods after it, which is the forecasting question itself. Ordinary K-fold CV is not always wrong for time-ordered data: Bergmeir, Hyndman and Koo (2018) show that it is valid for purely autoregressive models whose errors are uncorrelated. The claims triangle is not such a case, for the reasons given in Section 4.3.

The pooled test error (2.5) has the form of (3.4) with $m=2$ origins, $\tau_1=15$ and $\tau_2=18$. Al-Mudafer *et al.* (2021) minimise it over the hyperparameters, as in the second line of (3.4); the code evaluates the first line only.

3.5 SUMMARY

K-fold CV splits the data into disjoint folds that cover every observation once, fits the model once without each fold, and makes its choice on the average of the K held-out losses. Its early-stopping version for the bCCNN would average K validation curves over random sets of cells. Time-series CV keeps the averaging but replaces the random folds by forecast origins, with expanding training sets and later test sets. Chapter 4 compares the procedure of Chapter 2 with both.

CHAPTER 4

THE ROLLING ORIGIN COMPARED WITH K-FOLD CROSS-VALIDATION

4.1 INTRODUCTION

Chapters 2 and 3 described the procedure of the code and the two CV designs. This chapter compares them property by property, proves that the K-fold design (3.3) is infeasible for the bCCNN, places the claims split of Paper C and Härkönen (2021) in the same frame, and gives the procedure its proper name.

4.2 PROPERTY BY PROPERTY

4.2.1 The held-out sets overlap

Property (K1) fails. The test sets of the two test partitions share every cell of calendar periods 19 and 20 inside the 15×15 square. Diagonal k of a $\tau \times \tau$ square has $2\tau - k$ cells for $k \geq \tau$ (companion IV, (2.2)), so

$$|T_{15} \cap T_{18}| = \sum_{k=19}^{20} (2 \cdot 15 - k) = 11 + 10 = 21. \quad (4.1)$$

These 21 cells are also validation cells of the final partition, since they have $k \in \{19, 20\}$ and $i, j \geq 5 > e$: they are held out three times. Further, 31 of the 33 test cells of $\tau = 18$ are validation cells of $\tau = 20$, and the cell $(6, 2)$ is a replacement validation cell of both $\tau = 18$ and $\tau = 20$.

The held-out cells of one partition are, moreover, training cells of a later one. All 23 validation cells of $\tau = 15$ are training cells of $\tau = 18$ and of $\tau = 20$; of the 60 test cells of $\tau = 15$, 14 are training, 25 validation and 21 test cells of $\tau = 18$; and 28 of the 29 validation cells of $\tau = 18$ are training cells of $\tau = 20$. In K-fold CV a cell is held out in exactly one fit and used for fitting in all the others; here a cell can be held out, then trained on, or held out repeatedly.

4.2.2 The held-out sets do not cover the triangle

Property (K2) fails. The set H of all cells that are held out by some partition, as validation or as test cells, is

$$\begin{aligned} H &= (V_{15} \cup T_{15}) \cup (V_{18} \cup T_{18}) \cup V_{20}, \\ |H| &= 100 < 210 = |D|. \end{aligned} \quad (4.2)$$

The 110 cells that are never held out are of two kinds. Every cell with $k \leq 13$ is a training cell of every partition, except the four replacement cells $(5, 2)$, $(10, 2)$, $(6, 2)$ and $(12, 2)$: 87 of the 91 cells of the first 13 calendar periods are never held out. The other 23 lie in later calendar periods, all in the border $i \leq 2$ or $j \leq 2$, which the validation diagonals exclude by construction (companion IV, Section 2.4, property (c)) and which the test sets reach only in part. The early calendar periods are not what the design is meant to test. A K-fold CV would hold out each of the 210 cells exactly once.

4.2.3 The roles are ordered in time

Property (K3) fails. The three partitions cannot be permuted: partition τ uses only cells with $k \leq \tau$ for fitting and choosing, and its test cells all have $k > \tau$. The observed triangles are nested, $D_{15} \subset D_{18} \subset D_{20} = D$, and the training sets nearly so: $U_{15} \mid U_{18} = \{(6, 2), (12, 2)\}$ and $U_{18} \mid U_{20} = \{(14, 2)\}$, the replacement cells that move. Above all, the partitions have different jobs (Table 2.2): the final partition chooses the reported number of steps, and the other two only score.

4.2.4 The choice is not averaged

Property (K4) fails for the choice that matters. The code chooses the number of steps from one curve, the K-fold version from an average of curves:

$$t^* = \arg \min_t D_{V,20}(t) \text{ against } \hat{t} = \arg \min_t \sum_{k=1}^K D(y, \mu(\theta_t^{(-k)}); F_k). \quad (4.3)$$

The one average that the code does compute, $\hat{D}$ in (2.5), is computed after all choices are made and is not used to make any. It is an assessment of the procedure, in the sense of the three-way division of the data into training, validation and test sets, and not a selection criterion.

4.2.5 The held-out cells are not like the rest

Property (K5) is deliberately replaced. The validation cells of every partition are its latest diagonals, and the test cells lie after the valuation date; neither is a random sample of the triangle. The design assumes that the step count that predicts the next two calendar periods well at $\tau = 20$ also predicts the lower triangle well, which is the time-series assumption of Section 3.4 rather than exchangeability.

4.2.6 Summary of the comparison

Table 4.1 collects the comparison and adds the time-series CV of Section 3.4.

Table 4.1 The procedure of the code against K-fold and time-series cross-validation

Property	K-fold CV	Time-series CV	Rolling origin in the code
(K1) Held-out sets disjoint	Yes	No	No: 21 cells are test cells of both test partitions
(K2) Every observation held out once	Yes	No	No: 100 of 210 cells ever held out, some three times
(K3) Folds interchangeable	Yes	No: ordered origins	No: ordered, and different roles
(K4) Choice made on an average over fits	Yes	Yes	No: t^* from the final partition alone; the average $\hat{D}$ chooses nothing
(K5) Held-out cells exchangeable with the rest	Assumed	No: later in time	No: latest diagonals and later periods
Training sets	Overlapping, each of size $(K-1)/K$	Nested, expanding	Nested up to two replacement cells
Number of fits	K , then one refit	m , then one refit	Three runs, two refits for scoring, one final fit
Randomness of the split	Random folds	None	None

4.3 WHY K-FOLD CROSS-VALIDATION OVER THE CELLS IS INFEASIBLE

The K-fold design (3.3) is not only different from the code: it cannot be run with the chain-ladder start.

Proposition. For every $K \geq 2$ and every split of D into K folds satisfying (K1) and (K2), at least one of the fits in (3.3) has no training cell in accident period n or no training cell in development period n , so that its ccODP start is not defined.

Proof. Accident period n has the single observed cell $(n, 1)$, and development period n the single cell $(1, n)$. By (K2) the cell $(n, 1)$ lies in some fold F_k , and then $D \setminus F_k$ contains no cell of accident period n . The log-likelihood of the ccODP on $D \setminus F_k$ does not depend on α_n , so α_n is not identified and the start $\theta_0(\hat{\vartheta}(D \setminus F_k))$ has no value for the embedding of accident period n ; in the code `fit_odp_glm()` stops when a coefficient is NA (companion I, Section 2.6). The same argument applies to $(1, n)$ and β_n . $\square$

The proof uses only (K2), so no rearrangement of the folds, random or stratified, avoids it; at best both cells fall in the same fold. Accident period $n-1$ and development period $n-1$, with two cells each, are lost whenever both cells fall in one fold. The rolling origin avoids the problem by design: the cells of the latest accident and development periods of every partition, $(\tau, 1), (\tau-1, 1), (\tau-1, 2)$ and their mirror images, are training cells (companion IV, Section 2.4, property (c)).

The problem would remain even if the start were not needed. Paper C rules out holding out cells because the triangle is too small, "one of the reasons being that we only have $(I+1)I/2 = 78$ observations in the

upper triangle" (Gabrielli *et al.*, 2018: 11), and splits the claims instead. And a K-fold estimate on cells would measure the wrong thing. Figure 5.1(b) shows that each fold holds out cells from every calendar period; under the ccODP's assumption of independent cells, $CV(t)$ estimates how well the network fills in missing cells *inside* the observed calendar periods, whereas the reserve consists of calendar periods 21 to 39, all outside them.

4.4 THE CLAIMS SPLIT AND TWO-FOLD CROSS-VALIDATION

The alternative early-stopping signal of the code, `validation: claims_split`, splits the individual claims into two halves and builds a training triangle D^T and a validation triangle D^V from them (companion II, Section 2.3). The number of steps is the minimiser of the deviance on D^V along a run on D^T , and the reported model is refitted on the whole triangle. This is a single hold-out, not CV: one split, one direction, no average.

Härkönen (2021) runs the second direction as well. She found "that the machine learning models can be sensitive to the division of training and validation sets" and "fitted the models also by flipping the training and validation sets, i.e. we trained the model with our original validation data and then validated on the training data" (Härkönen, 2021: 33). Because the biases of the two directions often had opposite signs, this "gives rise to selecting the average outstanding reserve as the prediction" (Härkönen, 2021: 34). In terms of Section 3.2 the two halves are two folds that satisfy (K1) to (K3), and the claims are split at random, so (K5) holds as well: the two halves are independent copies of the same triangle under the model. What differs is (K4): each direction chooses its own number of steps and produces its own reserve, and it is the two *reserves* that are averaged, not the two validation curves. The 'VT' fits are therefore a two-fold arrangement used to average predictions, closer to an ensemble than to a cross-validated choice.

A K-fold version of the claims split is conceivable, with K groups of claims instead of two, but each fit would then train on $(K-1)/K$ of the volume and validate on $1/K$ of it, and the ccODP intercept, which carries the volume, would differ between the training and the validation triangles. Paper C uses two halves of equal size for this reason (companion II, Section 2.3.2). The code does not implement it.

4.5 WHAT THE PROCEDURE SHOULD BE CALLED

The procedure of the code combines two designs, and it is clearest to name them separately.

The choice of the number of steps is an out-of-time hold-out: a single validation set, the latest two diagonals of the observed triangle without the border, plus two replacement cells, on which one early-stopping run is scored. It is the time-ordered counterpart of the single hold-out of the claims split. Lindholm, Verrall, Wahl and Zakrisson (2020) describe the underlying three-way division of the data into a part for training, a part for determining the number of steps of the optimisation and a part for out-of-sample validation of the trained model.

The assessment is a rolling-origin evaluation with two forecast origins, $\tau = 15$ and $\tau = 18$: the time-series CV of Section 3.4, used for assessment only. Each origin repeats the whole procedure, including its own hold-out choice of the number of steps, so the test error assesses the procedure rather than one fitted network.

The whole is best described as rolling-origin evaluation with an out-of-time hold-out for early stopping. It is not K-fold CV: no property among (K1) to (K4) holds, and (K5) is replaced on purpose.

4.6 SUMMARY

The rolling origin violates every defining property of K-fold CV: its held-out sets overlap, they cover fewer than half of the observed cells, the partitions are ordered and have different roles, and the reported number of steps is chosen on one partition without averaging. A K-fold CV over the cells is, in addition, infeasible with the chain-ladder start, because the single cells of the latest accident and development periods must each be held out once. The claims split is a single hold-out, and the reversed split of Härkönen (2021) is a two-fold arrangement that averages reserves. The test partitions form a time-series CV with two origins, used for assessment only. Chapter 5 illustrates these counts.

CHAPTER 5

NUMERICAL ILLUSTRATION AND CONCLUSIONS

5.1 INTRODUCTION

Chapter 4 derived the overlaps and the coverage of the partitions and the infeasibility of K-fold CV over the cells. This chapter shows them in the replication of the split for the configuration of the code, contrasts the two designs in two figures, and states the conclusions.

5.2 WHERE THE HELD-OUT CELLS GO

Table 5.1 follows the held-out cells of each partition into the later partitions. Each row adds up to the size of the set in the first column.

Table 5.1 Roles in later partitions of the cells held out by each partition

Cells	Number	Training at 18	Validation at 18	Test at 18	Training at 20	Validation at 20
Validation at 15, V_{15}	23	23	0	0	23	0
Test at 15, T_{15}	60	14	25	21	39	21
Validation at 18, V_{18}	29	–	–	–	28	1
Test at 18, T_{18}	33	–	–	–	2	31

The two test cells of $\tau = 18$ that are training cells at 20 are $(2, 18)$ and $(18, 2)$, which lie in the border. Over all three partitions, 110 cells are never held out, 43 are held out once, 36 twice and 21 three times; 31 cells, the validation cells of the final partition except the two replacement cells, are never used for fitting in any partition.

Figure 5.1 contrasts the two designs along calendar time. For each calendar period k , a bar shows the share of the cells of that diagonal of D in each role. In panel (a), the rolling origin, the validation cells sit on the latest two diagonals of each partition, the test cells after τ , and the cells outside the partition's square are unused. In panel (b), a random five-fold split of the cells (seed 2026), every fold holds out about a fifth of every calendar period, early and late alike.

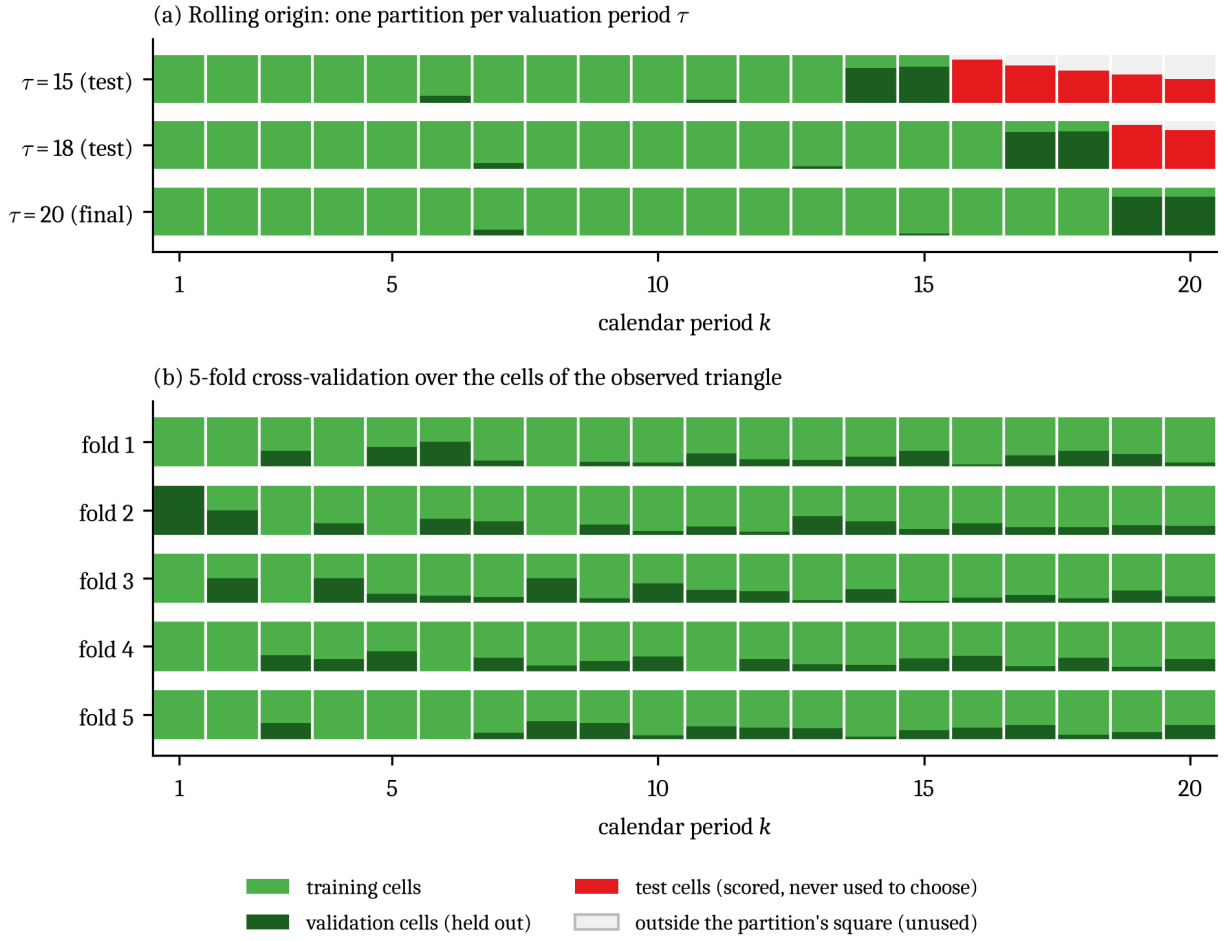


Figure 5.1 Roles of the cells of each calendar period: rolling origin against five-fold cross-validation

Figure 5.2 shows the same contrast over the triangle. Panel (a) gives the fold of each cell in the five-fold split: every cell is in exactly one fold, and the framed cells $(20, 1)$ and $(1, 20)$ fall in folds 1 and 3, whose fits therefore lose accident period 20 and development period 20 respectively, as the proposition of Section 4.3 requires. Panel (b) gives the number of times each cell is held out by the rolling origin: zero in the early calendar periods and the border, up to three on the diagonals 19 and 20.

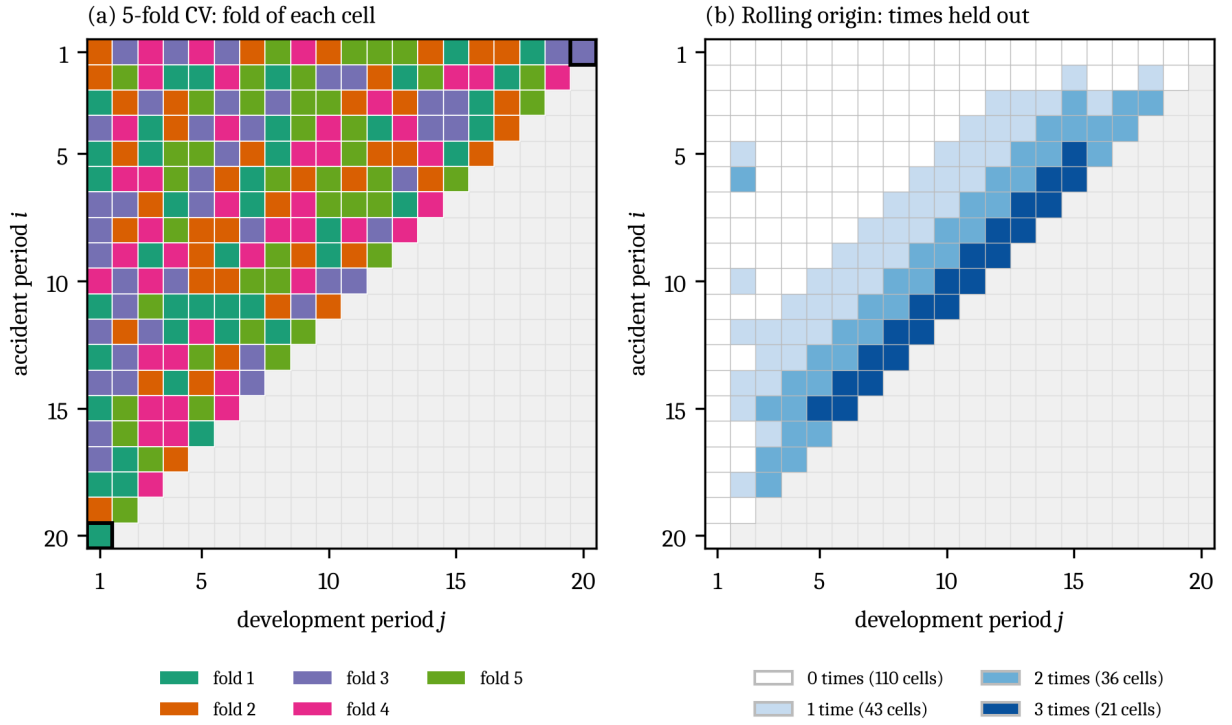


Figure 5.2 Held-out cells over the triangle: five-fold cross-validation against the rolling origin

The replication also drew 2 000 further random five-fold splits of the 210 cells. In every one of them at least one fold left an accident period or a development period without a training cell, which is the proposition of Section 4.3 observed rather than proved.

5.3 SUMMARY AND CONCLUSIONS

The three rolling-origin partitions of the bCCNN are not the folds of a K-fold CV. Each partition fits the ccODP on its training cells, runs the network and chooses its own number of steps on its own latest diagonals. Only the final partition's number of steps reaches the reported model; the two test partitions use theirs to replay the procedure at the earlier valuation dates 15 and 18 and score it on the 60 and 33 cells observed afterwards, and their scores are pooled into a test error that the code reports and never minimises.

Compared with K-fold CV, the held-out sets overlap (21 cells are test cells of both test partitions), they cover 100 of the 210 observed cells, the partitions are ordered in time and have different roles, and no choice is made on an average. A K-fold CV over the cells could not be run at all with the chain-ladder start, because the latest accident and development periods consist of one cell each. The design is a rolling-origin evaluation with two forecast origins, the time-series CV of the forecasting literature used for assessment only, combined with an out-of-time hold-out for early stopping. The claims split is a single hold-out, and the reversed split of Härkönen (2021) is a two-fold arrangement that averages reserves rather than validation scores.

Two consequences follow for the use of the code. First, the reported number of steps rests on one validation set of 33 cells and one seed; averaging the validation curves of several seeds, or of the three partitions, would be a change of method, not a property of the present one. Second, the pooled test error is an honest out-of-time assessment precisely because nothing is chosen on it; if hyperparameters were selected on it, as Al-Mudafer *et al.* (2021) do, it would become a validation score and would no longer measure the performance of the selected model without bias. The entry `cv: folds: 5` in `config.yml` is not used and could be removed or documented as unused.

References

- Al-Mudafer, M.T., Avanzi, B., Taylor, G. & Wong, B. 2021. Stochastic loss reserving with mixture density neural networks. arXiv preprint 2108.07924.
- Bergmeir, C., Hyndman, R.J. & Koo, B. 2018. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70-83.
- Department of Statistics and Actuarial Science. 2026. *Research assignment guide*. Last updated 28 January 2026. Stellenbosch: Stellenbosch University.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2020. Neural network embedding of the over-dispersed Poisson reserving model. *Scandinavian Actuarial Journal*, 2020(1):1-29.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2018. Neural network embedding of the over-dispersed Poisson reserving model. SSRN working paper 3288454, version of 21 November 2018.
- Härkönen, V. 2021. On claims reserving with machine learning techniques. Unpublished master's thesis. Stockholm: Stockholm University.
- Hastie, T., Tibshirani, R. & Friedman, J. 2009. *The elements of statistical learning: data mining, inference, and prediction*. 2nd edition. New York: Springer.
- Hyndman, R.J. & Athanasopoulos, G. 2021. *Forecasting: principles and practice*. 3rd edition. Melbourne: OTexts.
- Lindholm, M., Verrall, R., Wahl, F. & Zakrisson, H. 2020. Machine learning, regression models, and prediction of claims reserves. *Casualty Actuarial Society E-Forum*, Summer 2020:1-47.
- Tashman, L.J. 2000. Out-of-sample tests of forecasting accuracy: an analysis and review. *International Journal of Forecasting*, 16(4):437-450.

APPENDIX A

LIST OF SYMBOLS

The symbols in the upper block are those of companions I, II and IV, and those in the lower block are new in this report; the third column gives the section where each symbol is introduced.

Table A.1 Symbols used in this report

Symbol	Meaning	Section
n	Size of the triangle, $n = 20$	2.2
i, j, k	Accident period, development period, calendar period $k = i + j - 1$	2.2
D, F	The observed triangle (210 cells); the lower triangle, whose sum is the reserve	2.2
τ	Valuation period of a partition, $\tau \in \{15, 18, 20\}$	2.2
$D_\tau, U_\tau, V_\tau, T_\tau$	Cells observed at τ ; training, validation and test cells of the partition	2.2
$D(y, \mu; S)$	Poisson deviance on the cells S	2.3
$\hat{\vartheta}(S), \theta_0(\hat{\vartheta})$	ccODP fitted on the cells S ; the ccODP start built from it	2.3
$T_t^s(\theta, S)$	t RMSprop steps with seed s on the cells S from θ	2.3
T_{max}	Length of each early-stopping run, 1 000	2.3
$D_{V,\tau}(t), t_\tau^*$	Validation deviance of partition τ after t steps; its minimiser	2.3
$\dot{D}$	Pooled test error over the test partitions	2.5
$t^*, \hat{\theta}, \hat{R}$	Number of steps, parameters and reserve of the reported model	2.6
	New in this report	
$\theta_t^{(\tau)}$	Network of partition τ after t steps	2.3
H	Cells held out by at least one partition	4.2.2
$\theta_\tau^{refit}, S_\tau$	Network refitted at τ for t_τ^* steps; its test deviance on T_τ	2.4
I, K, F_k	Index set of the observations; number of folds; fold k	3.2

Symbol	Meaning	Section
$\lambda, CV(\lambda), \hat{\lambda}$	Tuning parameter; its K-fold CV score; the minimiser	3.2
$\theta_t^{(-k)}, CV(t), \hat{t}$	Network trained without fold k ; K-fold score of the number of steps; its minimiser	3.3
$\tau_1, \dots, \tau_m, TSCV(\lambda)$	Forecast origins; the time-series CV score	3.4
(K1) to (K5)	The defining properties of K-fold CV	3.2